
CSE D 501

*Regularization and Dimensionality
Reduction*

Taylor Kessler Faulkner

University of Washington

October 8, 2025



Today's Class

- Gradient descent example, Bias/Variance recap, Intro to feature standardization/data representation
- Break
- Regularization in regression
- Break
- Dimensionality reduction

Admin

- New submission protocol for Assignment 1
 - Only upload .ipynb file, not python file
 - Name your file assignment1_[last_name]_[first_name].ipynb
 - Mine would be assignment1_kesslerfaulkner_taylor.ipynb
 - Just upload single assignment1_[last_name]_[first_name].ipynb file, no need for zip file

Gradient Descent

Convex function when parameters are linear

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

$\nabla L(w)$: the gradients of loss function L
(ex: MSE) on a set of weights w

I.e., each w_i is updated as

$$w_i^{(t+1)} = w_i^{(t)} - \eta \frac{dL(w_i)}{dw_i}$$

Convergence: $w^{(t+1)} - w^{(t)} < \epsilon$

Gradient Descent requires **convex** function to converge to optimal solution

Gradient Descent Example

Use the following linear regression model:

$$\hat{y} = w_0 + w_1 x$$

You have two data points: (3, 5), (5, 7)

Start with $w_0 = w_1 = 0$. Use MSE as the loss function

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Use learning rate $\eta = 0.1$ and perform one step of gradient descent to update w_1

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Gradient Descent Example

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Start by taking the *partial derivative* of MSE with respect to w_1

$$\frac{\delta MSE}{\delta w_1} = \frac{\delta}{\delta w_1} \left(\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \right)$$

Where are w_0 and w_1 in this equation?

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Gradient Descent Example

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Start by taking the *partial derivative* of MSE with respect to w_1

$$\frac{\delta MSE}{\delta w_1} = \frac{\delta}{\delta w_1} \left(\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \right)$$

Where are w_0 and w_1 in this equation?

$$\frac{\delta MSE}{\delta w_1} = \frac{\delta}{\delta w_1} \left(\frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_i - y_i)^2 \right)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Gradient Descent Example

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Start by taking the *partial derivative* of MSE with respect to w_1

$$\frac{\delta MSE}{\delta w_1} = \frac{\delta}{\delta w_1} \left(\frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_i - y_i)^2 \right)$$

What does partial derivative mean? Taking the derivative of a function by letting one variable change and holding the others constant

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Gradient Descent Example

Start by taking the *partial derivative* of MSE with respect to w_0 and w_1

$$\begin{aligned}\frac{\delta MSE}{\delta w_1} &= \frac{\delta}{\delta w_1} \left(\frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_i - y_i)^2 \right) \\ &= \frac{1}{n} \sum_{i=1}^n \frac{\delta}{\delta w_1} (w_0 + w_1 x_i - y_i)^2 \\ &= \frac{1}{n} \sum_{i=1}^n [2(w_0 + w_1 x_i - y_i) * \frac{\delta}{\delta w_1} (w_0 + w_1 x_i - y_i)] \\ &= \frac{2}{n} \sum_{i=1}^n (w_0 + w_1 x_i - y_i) * x_i\end{aligned}$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $\mathbf{w}^{(0)}$ (can be random)
- Step size η

Gradient Descent Example

Now plug in $w_0 = w_1 = 0$ (the initial values) and the $n = 2$ data points $(3, 5), (5, 7)$ to obtain

$$\begin{aligned}\frac{\delta MSE}{\delta w_1} &= \frac{2}{n} \sum_{i=1}^n (w_0 + w_1 x_i - y_i) * x_i \\ &= (((0 + 0 * 3) - 5) * 3) + (((0 + 0 * 5) - 7) * 5) \\ &= (-5 * 3) + (-7 * 5) \\ &= -50\end{aligned}$$

With $\eta = 0.1, w_1^{(t)} = 0$:

$$w_1^{(t+1)} = 0 - 0.1(-50) = 5$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w})$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Bias/Variance Recap: Model Error Types

- Bias
- Variance
- Irreducible error
 - We can't control irreducible error: comes from uncontrollable aspects such as noisy observations
 - Example: data from a US census may be noisy based on self-reported information

Bias

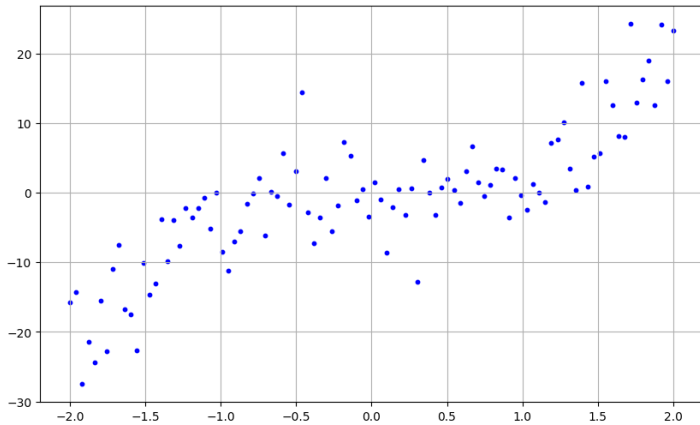
- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$\mathbb{E}[\hat{f}(x)] - f(x)$$

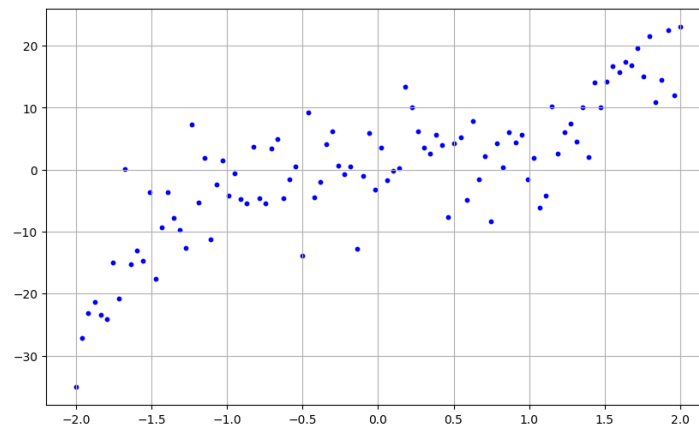
- Can't compute the true bias, because we don't have access to the true function $f(x)$, but we can estimate using a test set
- Less complex models tend to have higher bias, as they cannot capture the complexity/patterns in the data

Bias

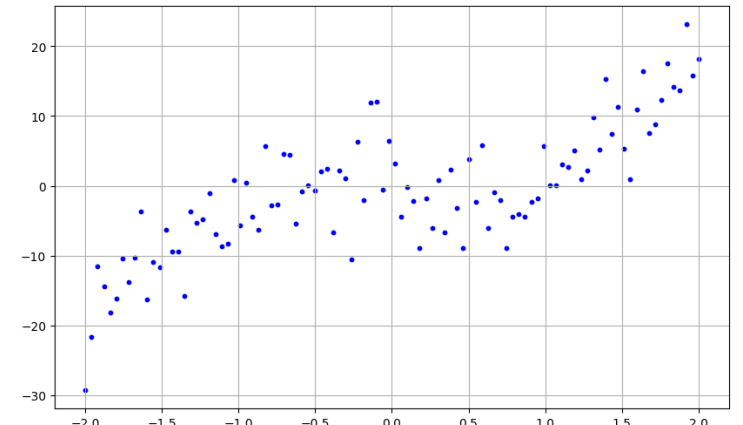
- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict
- Where does the randomness come from for computing $\mathbb{E}[\hat{f}(x)]$?
 - The dataset
 - Could be subsets of one large dataset (all shown below are $3x^3$ with noise, but different data points)



Dataset 1



Dataset 2

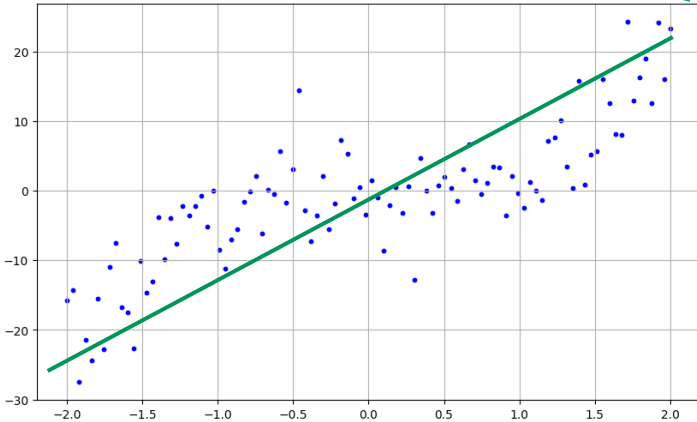


Dataset 3

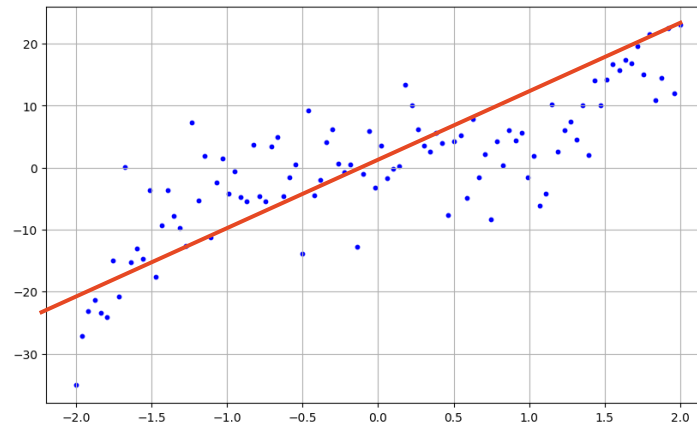
Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

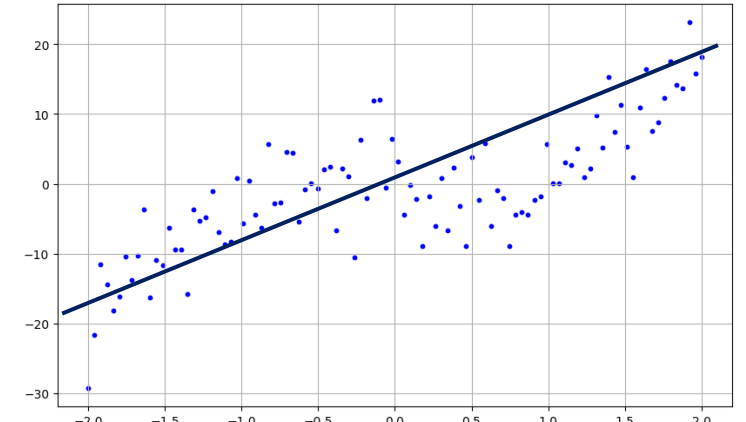
$$\mathbb{E}[\hat{f}(x)] - f(x)$$



Dataset 1



Dataset 2

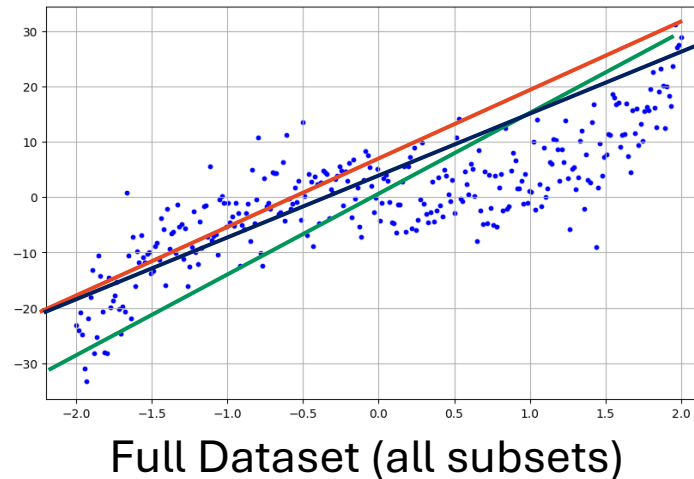


Dataset 3

Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

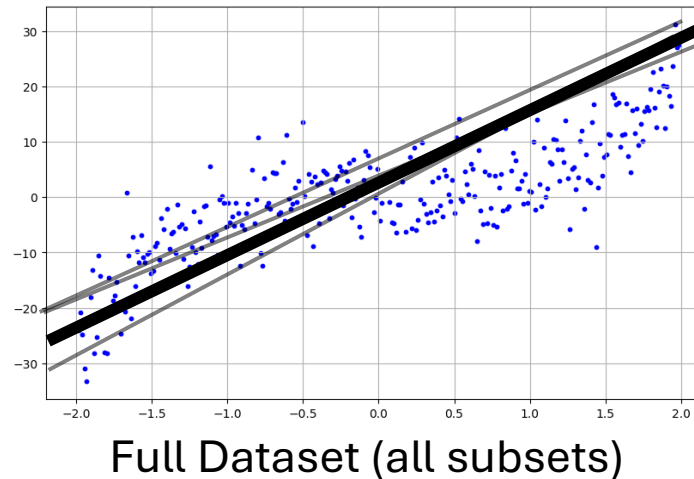
$$\mathbb{E}[\hat{f}(x)] - f(x)$$



Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$\mathbb{E}[\hat{f}(x)] - f(x)$$

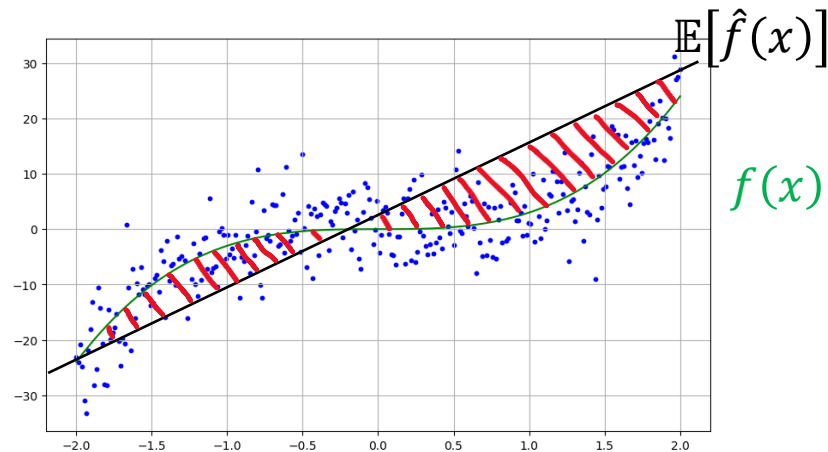


Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$\mathbb{E}[\hat{f}(x)] - f(x)$$

← Could estimate with test set



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

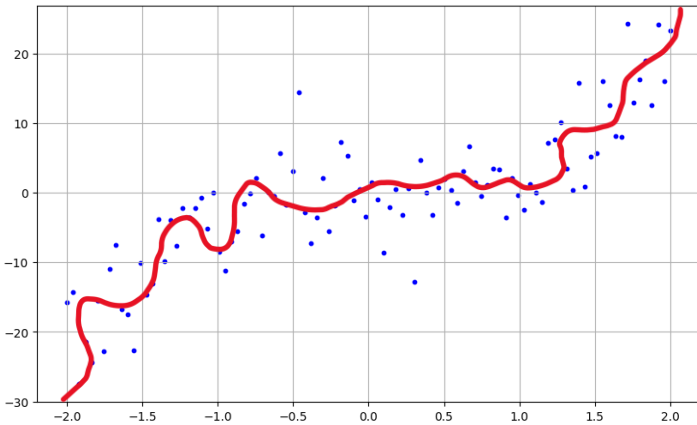
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

- Could test by running a model M on subsets of the data, finding the average of the predictions, and comparing that average to each individual prediction
- More complex models tend to have higher variance, as they fit more closely to the training data, causing high variations in learned parameters

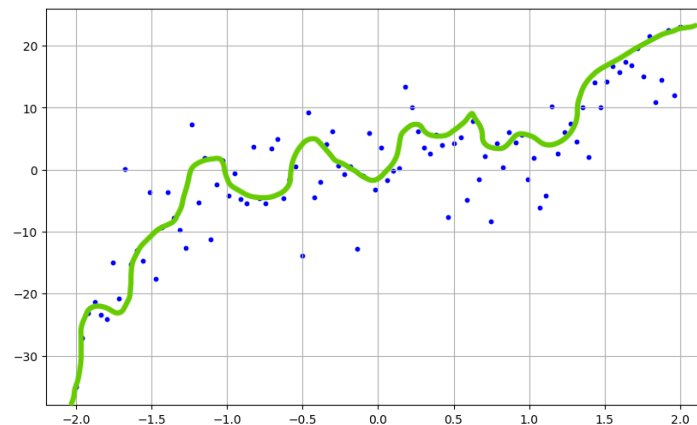
Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

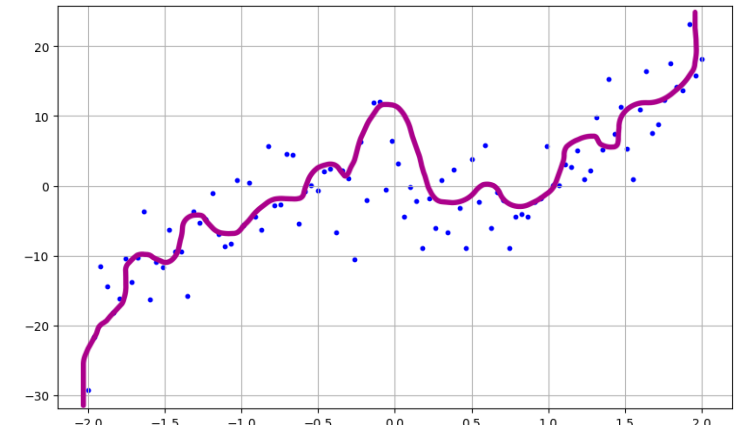
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Dataset 1



Dataset 2

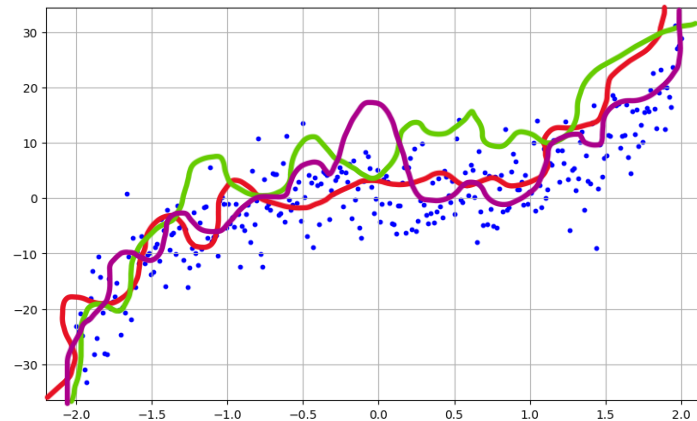


Dataset 3

Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

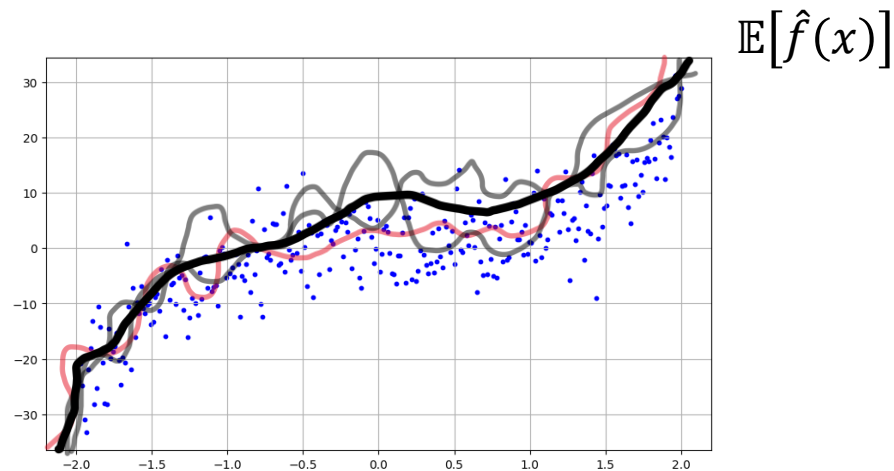
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

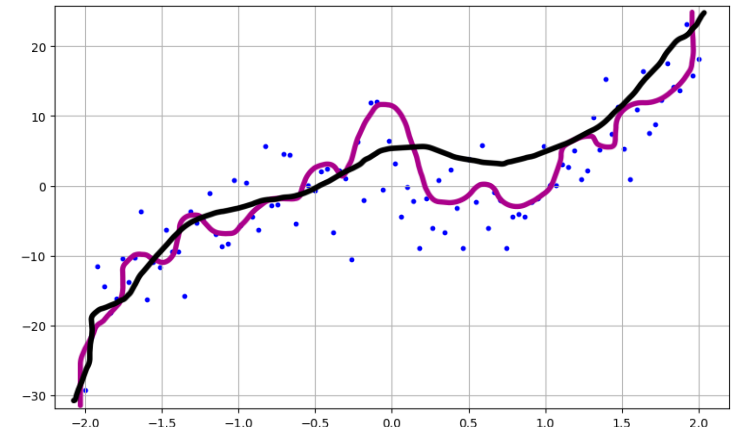
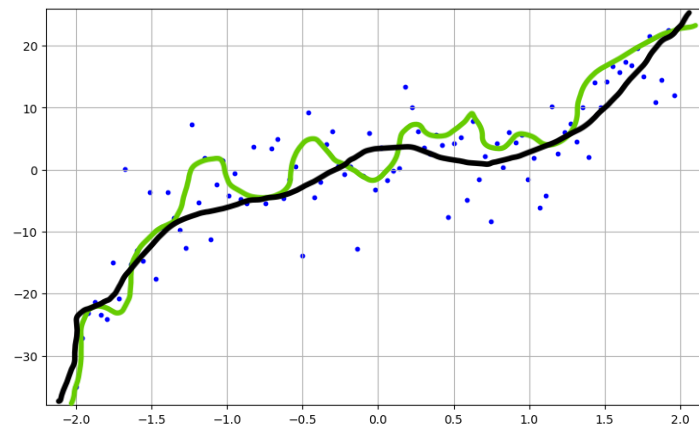
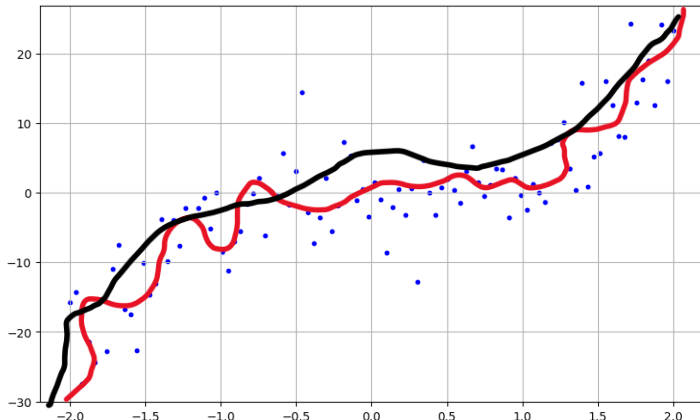
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Average of differences squared

Bias-Variance Tradeoff

- Simpler models tend to have **high bias** and **low variance**
- Complex models tend to have **low bias** and **high variance**

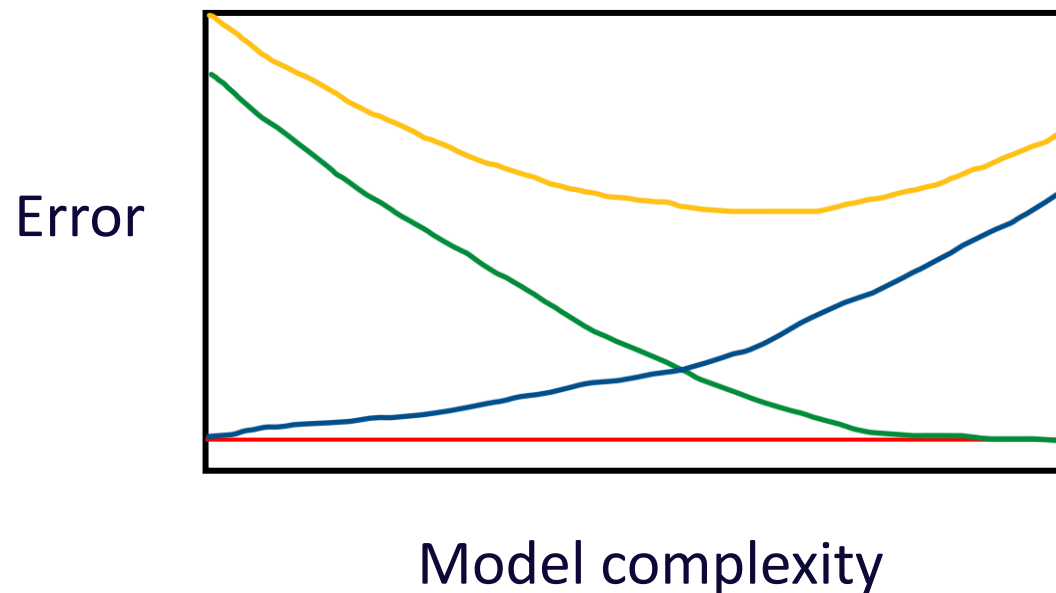
$$\mathbb{E} \left[\left(y - \hat{f}(x) \right)^2 \right] = \left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] + \sigma_{\epsilon}^2$$

Total error = bias squared + variance + irreducible error

Bias-Variance Tradeoff

$$\mathbb{E} \left[\left(y - \hat{f}(x) \right)^2 \right] = \left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] + \sigma_{\epsilon}^2$$

Total error = bias squared + variance + irreducible error



Bias-Variance Tradeoff and Dataset Size

- As dataset size goes up, the variance will lower (less overfitting)
- Example: a degree 9 polynomial on 9 datapoints will be overfit
 - But on 10,000 datapoints, much less likely to overfit!
- Bias is not impacted by the dataset size, as bias averages across several sampled datasets
- Takeaway: this is one reason why deep learning needs huge datasets: to avoid overfitting with many trainable parameters!

Data Representation

- So far, we've only talked about our data in terms of raw inputs and features: we train a model on features, and predict an output
- But should we be training directly on the data we're given? Are there any other steps we need to take before training some of the models we'll use?
- First, let's talk about types of features

Types of Features

- Numeric: data described by a number
 - Discrete (cannot be subdivided)
 - E.g. number of bedrooms
 - Continuous (can be subdivided)
 - E.g. sq footage of the house
- Categorical: data described by a category
 - Ordinal (has an order)
 - E.g. nearby school quality (good/okay/poor)
 - Nominal (no order)
 - E.g. type of flooring (carpet, linoleum, hardwood)

Data Representation

- How do we deal with categorical data?
- Example 1:
 - You decide to include a feature noting the quality of nearby schools
 - You start with school quality = [poor, okay, good]
 - This feature is **ordinal**, but not numeric
 - Question: Ideas on representing this feature numerically?

Data Representation

- How do we deal with categorical data?
- Example 1:
 - You decide to include a feature noting the quality of nearby schools
 - You start with school quality = [poor, okay, good]
 - This feature is **ordinal**, but not numeric
 - Question: Ideas on representing this feature numerically?
 - Answer: You can assign each value to a number: school quality = [0, 1, 2]
 - Poor -> 0, okay -> 1, good -> 2

Data Representation

- How do we deal with categorical data?
- Example 2:
 - You notice that the type of flooring in a house affects the house price, and you want to encode that in a feature. You start with flooring = [carpet, hardwood, linoleum]
 - This data is **nominal**
 - Question: can we encode it the same way as the school quality?

Data Representation

- How do we deal with categorical data?
- Example 2:
 - You notice that the type of flooring in a house affects the house price, and you want to encode that in a feature. You start with flooring = [carpet, hardwood, linoleum]
 - This data is **nominal**
 - Question: can we encode it the same way as the school quality?
 - Answer: no, because that implies an ordering which misrepresents the data and may cause the function to be nonlinear
 - New way: **one-hot encoding**

Data Representation

- For one-hot encoding, you make each value its own binary feature
 - We'll use this on Assignment 2
- If you started with flooring = [carpet, hardwood, linoleum], you'd go to:

Flooring	House Price
Linoleum	\$500K
Hardwood	\$750K
Carpet	\$600K
Linoleum	\$700K



Flooring - Carpet	Flooring - Hardwood	Flooring - Linoleum	House Price
0	0	1	\$500K
0	1	0	\$750K
1	0	0	\$600K
0	0	1	\$700K

Coefficients and Standardization

- The magnitude of coefficients/weights/parameters in regression indicates how much changes in each feature contribute to the final value
- Question: can coefficient magnitudes be compared directly to each other to say how much each feature changes the output?

Coefficients and Standardization

- The magnitude of coefficients/weights/parameters in regression indicates how much changes in each feature contribute to the final value
- Question: can coefficient magnitudes be compared directly to each other to say how much each feature changes the output?
- Answer: not without standardization!
- Example: two features, x_1 = square footage of a house, and x_2 = distance in miles from nearest grocery store
 - Square footage will likely be in the high hundreds to thousands, distance in miles to nearest grocery store will likely be in the single digits
 - If $w_1 = 10$ and $w_2 = 100$, you'll still see a larger contribution to the output from x_1
 - Comparing the coefficients directly won't tell us the relative contribution

Coefficients in Regression

- Instead, we can standardize features so that they all have the same scale
- $x_{standardized} = \frac{x - \mu}{\sigma}$, where μ is the mean of the feature values and σ is the standard deviation of the feature values
- After features are standardized, we can compare the magnitude of coefficients!
- In code:

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler().fit(X)  
X_standardized = scaler.transform(X)
```

- Important: only apply fit() to the training set, not the full dataset. Then apply transform() to the whole dataset. Why?

Coefficients in Regression

- Instead, we can standardize features so that they all have the same scale
- $x_{standardized} = \frac{x - \mu}{\sigma}$, where μ is the mean of the feature values and σ is the standard deviation of the feature values
- After features are standardized, we can compare the magnitude of coefficients!
- In code:

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler().fit(X)  
X_standardized = scaler.transform(X)
```

- Important: only apply fit() to the training set, not the full dataset. Then apply transform() to the whole dataset. Why?
 - To avoid knowledge from the validation and test sets bleeding into your training set

Coefficients in Regression

- Coefficient magnitude can also give us a clue about overfit models (assuming standardized data)
- “Large” coefficients can indicate overfitting and higher variance, as small changes in the features can lead to wild swings in the output

Break

Get some water, stretch your legs, chat, be back in 10

Regularization in Regression

Note: Regularization is also applied to classification and neural networks/deep learning!

Regularization in Regression

- Last week, we discussed how adding features can contribute to overly complex models that **overfit**
- We can choose parameters by:
 - Choosing only parameters that are highly correlated with the target value
 - Not choosing parameters that are highly correlated with each other
 - Domain knowledge
- But it would save time and let us try out more features if we could instead train a model that was less likely to overfit
- Reducing the magnitude of coefficients (to small values or zero) for features can reduce complexity, variance, and overfitting
 - Question: while variance decreases, we may see what value increase?

Regularization in Regression

- Last week, we discussed how adding features can contribute to overly complex models that **overfit**
- We can choose parameters by:
 - Choosing only parameters that are highly correlated with the target value
 - Not choosing parameters that are highly correlated with each other
 - Domain knowledge
- But it would save time and let us try out more features if we could instead train a model that was less likely to overfit
- Reducing the magnitude of coefficients (to small values or zero) for features can reduce complexity, variance, and overfitting
 - Question: while variance decreases, we may see what value increase?
 - Answer: bias, due to the bias/variance tradeoff

Coefficients in Regression

- Standard Linear Regression uses a quality metric that minimizes loss

$$\hat{w} = \operatorname{argmin}_w L(w)$$

- Can we change this quality metric to also penalize large coefficients?
 - We already have $L(w)$, our measure of fit
 - We can add $R(w)$, a measure of the magnitude of coefficients

$$\hat{w} = \operatorname{argmin}_w L(w) + \lambda R(w)$$

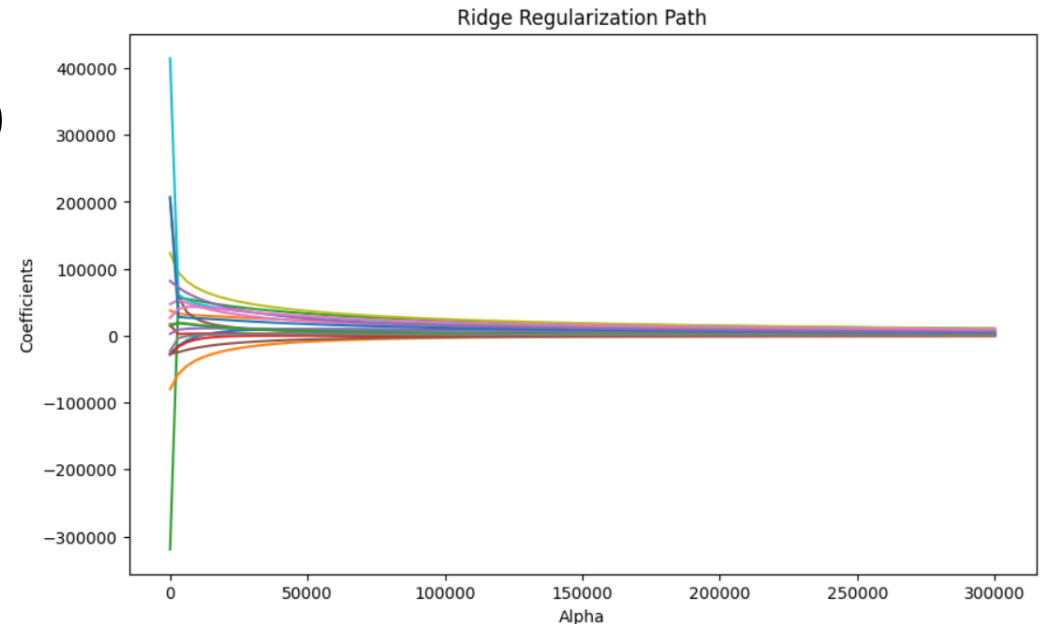
- λ is the *regularization parameter*, a hyperparameter that you can set
- $R(w)$ is some measure of how large *all* of the coefficients are. Commonly used:
 - L1 norm: $\|\mathbf{w}\|_1 = \sum_{i=1}^D |w_i|$
 - L2 norm: $\|\mathbf{w}\|_2^2 = \sum_{i=1}^D w_i^2$

Ridge Regression (L2 norm)

- Ridge Regression changes the quality metric to:

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \operatorname{MSE}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

- The hyperparameter λ controls how much importance is placed on regularization vs minimizing error, ranging from 0 to ∞
 - $\lambda = 0$ is just standard Linear Regression
 - $\lambda = \infty$ will force all parameters towards 0
- Question: How do we choose λ ? If λ is too small, we may overfit to the training set; if λ is too big, we will end up underfitting.

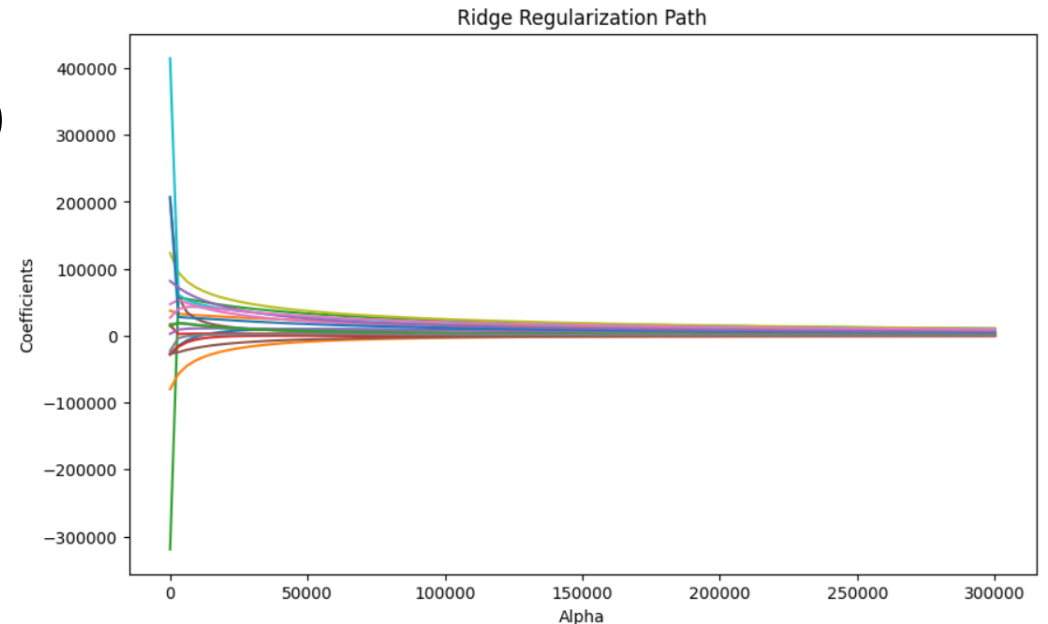


Ridge Regression (L2 norm)

- Ridge Regression changes the quality metric to:

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \operatorname{MSE}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

- The hyperparameter λ controls how much importance is placed on regularization vs minimizing error, ranging from 0 to ∞
 - $\lambda = 0$ is just standard Linear Regression
 - $\lambda = \infty$ will force all parameters towards 0
- Question: How do we choose λ ? If λ is too small, we may overfit to the training set; if λ is too big, we will end up underfitting.
- Answer: validation set or cross validation



Caveats to Ridge Regression (L2 norm)

- Question: when measuring error on the validation/test set, should you use

$$MSE(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

or

$$MSE(\mathbf{w})?$$

Caveats to Ridge Regression (L2 norm)

- Question: when measuring error on the validation/test set, should you use

$$MSE(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

or

$$MSE(\mathbf{w})?$$

- Answer: $MSE(\mathbf{w})$! The only thing that matters on future data is the accuracy of the predictions
- When estimating error on future unseen data, we want to only use the quality metric measuring error (in this case, MSE)
- $MSE(\mathbf{w})$ measures the error of predictions
- $MSE(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$ measures the error of predictions + coefficient size
- Use regularization during training, **not during prediction**

Caveats to Ridge Regression (L2 norm)

- Question: are there any parameters that you should not apply regularization to?
Think about what each parameter in this equation represents:

$$\hat{y} = w_0 + w_1x$$

Caveats to Ridge Regression (L2 norm)

- Question: are there any parameters that you should not apply regularization to? Think about what each parameter in this equation represents:

$$\hat{y} = w_0 + w_1x$$

- Answer: you don't regularize intercept/bias/ w_0 terms: a high intercept just means that the y value units might be high, and doesn't affect the curvature of the loss function

Caveats to Ridge Regression (L2 norm)

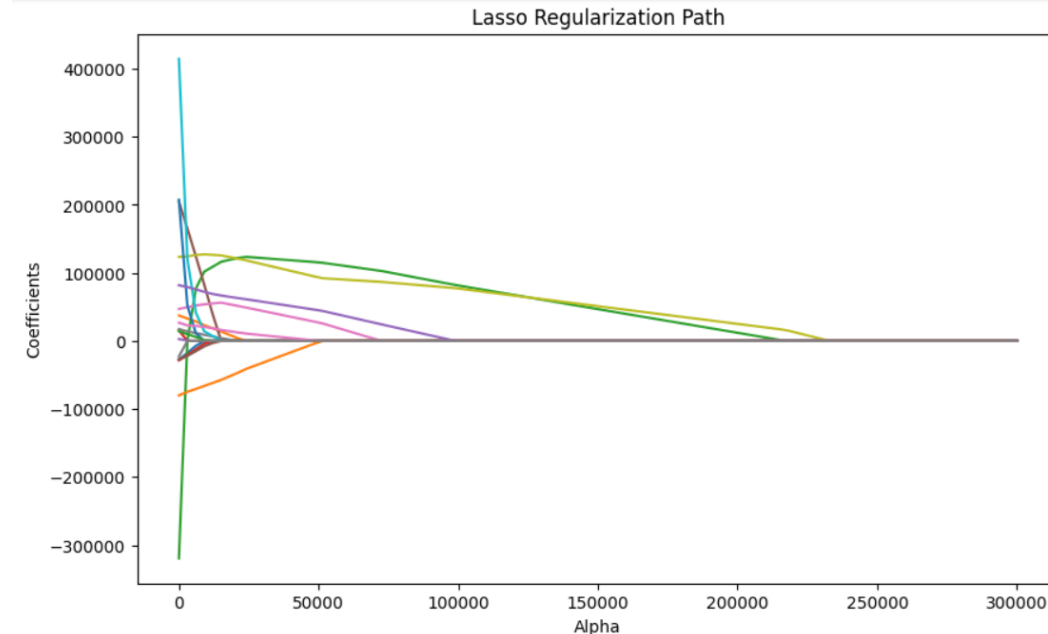
- Question: does Ridge Regression work as intended with unstandardized data?

Caveats to Ridge Regression (L2 norm)

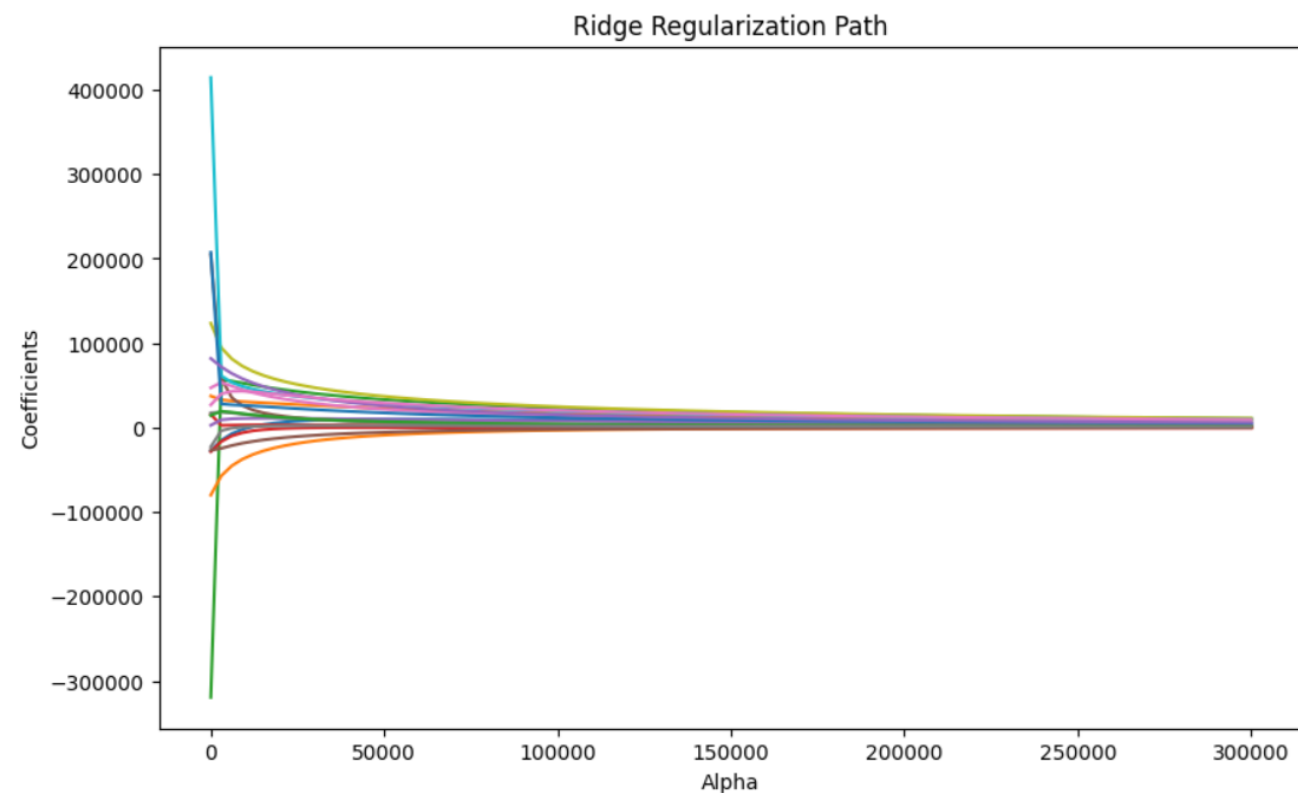
- Question: does Ridge Regression work as intended with unstandardized data?
- Answer: no, as we covered earlier, unstandardized data can lead to an inability to compare parameter magnitudes, as different features might have different scales

LASSO Regression (L1 norm)

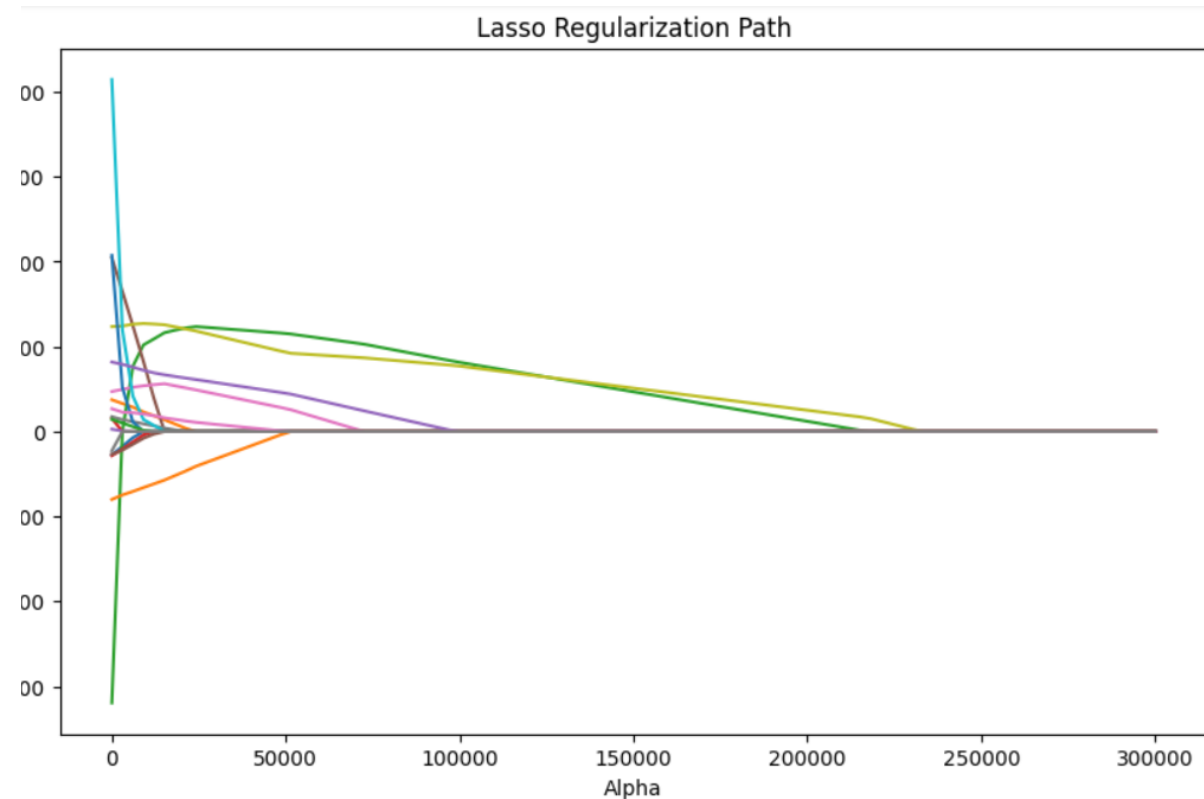
- LASSO Regression changes the quality metric to:
$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \operatorname{MSE}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$$
- The hyperparameter λ controls how much importance is placed on regularization vs minimizing error, ranging from 0 to ∞
 - $\lambda = 0$ is just standard Linear Regression
 - $\lambda = \infty$ will force all parameters towards 0
- Still use validation set or cross validation to choose λ
- LASSO causes more **sparsity** in the coefficient vector
- Sparsity: coefficients set to exactly 0



LASSO Regression and Sparsity



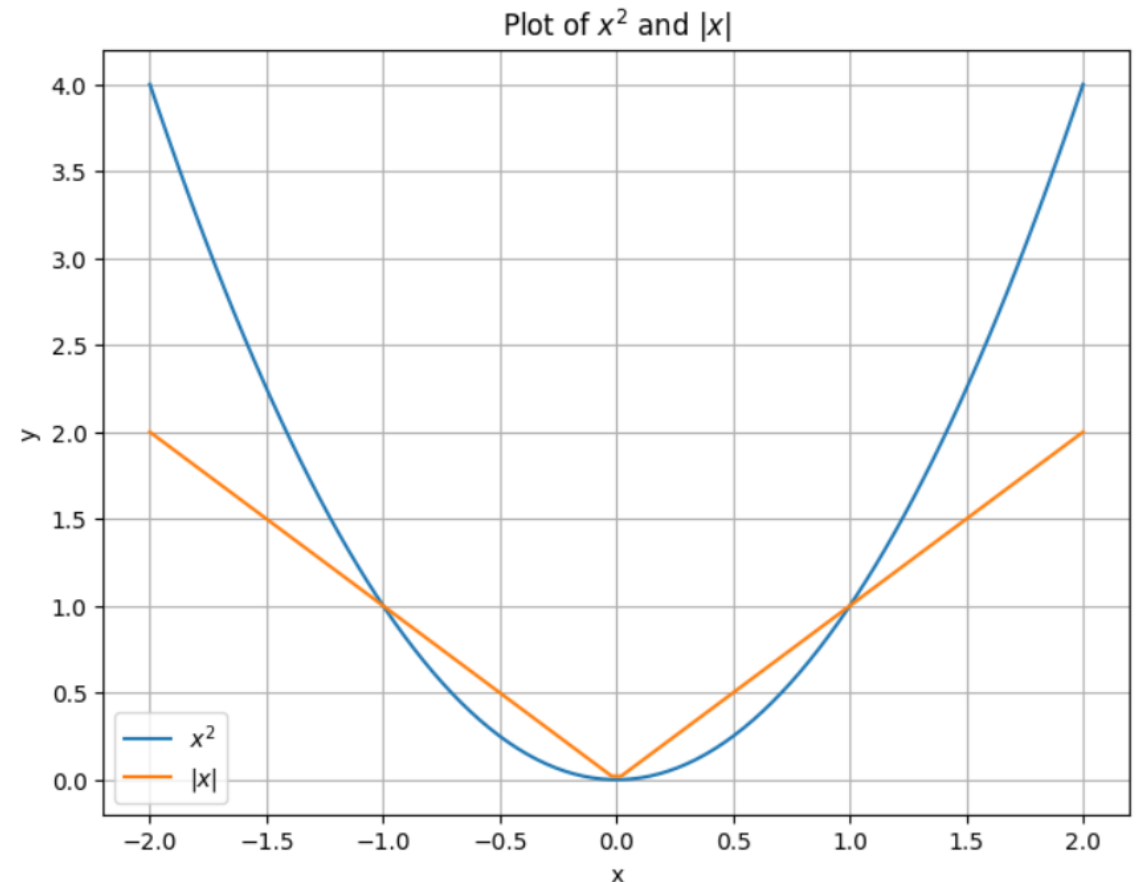
Ridge coefficients go *towards* 0, but are small values



LASSO coefficients go to exactly 0

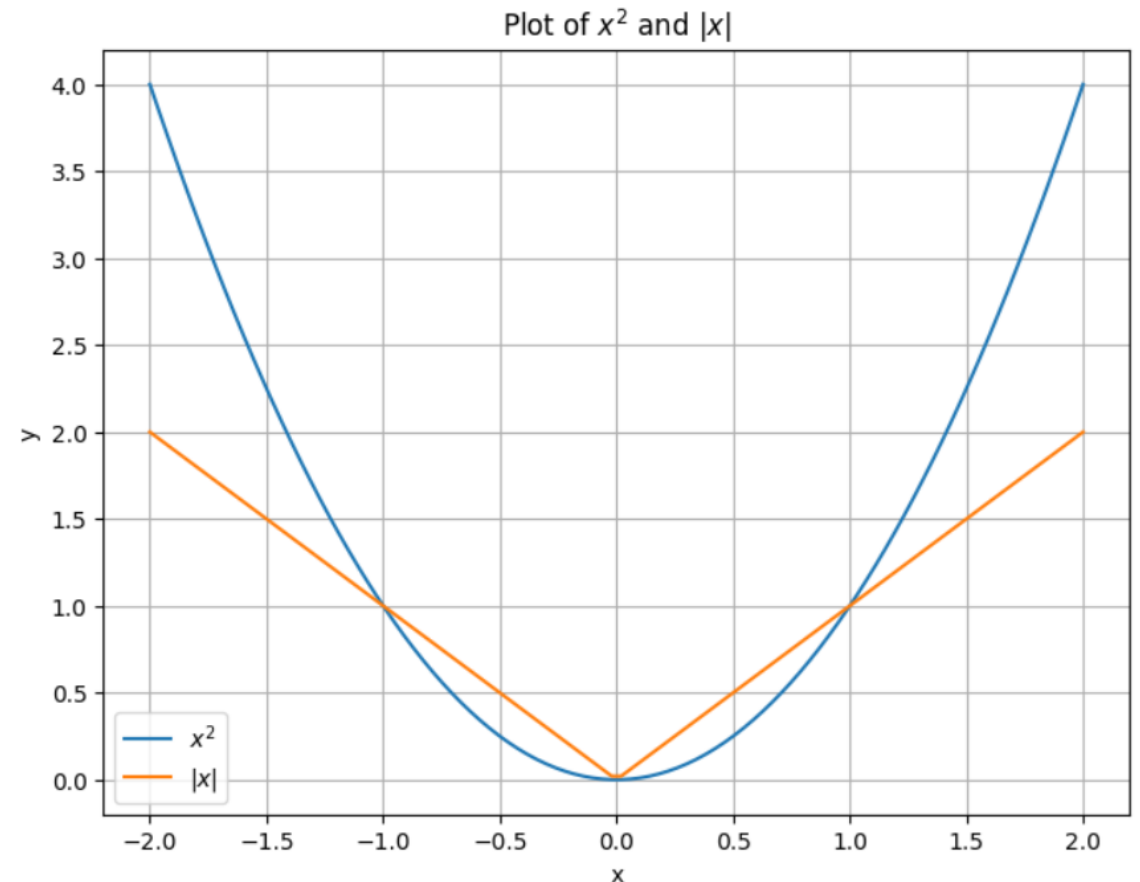
LASSO Regression and Sparsity

- When using the L1 norm as a regularizer in LASSO, it favors sparse solutions because of the shape of the norm
- We can compare to using the L2 norm as a regularizer in Ridge
- Question: when w_i is small (<1), what happens to w_i^2 ?



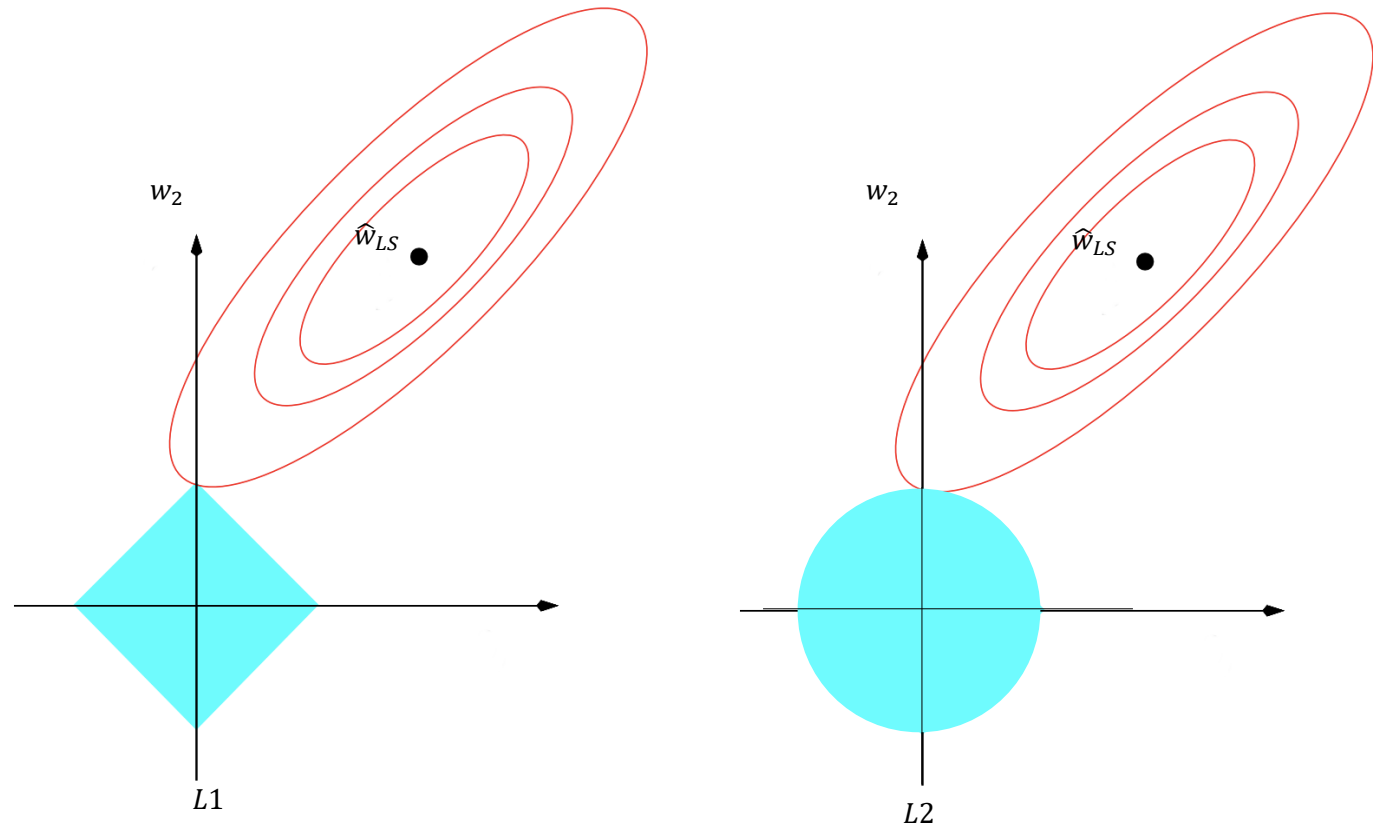
LASSO Regression and Sparsity

- When using the L1 norm as a regularizer in LASSO, it favors sparse solutions because of the shape of the norm
- We can compare to using the L2 norm as a regularizer in Ridge
- Question: when w_i is small (<1), what happens to w_i^2 ?
- Answer: it gets tiny ($w_i^2 < w_i$)! So coefficients in Ridge get close to 0, but not actually zero
- LASSO coefficients are penalized linearly all the way to 0



LASSO Regression and Sparsity

- LASSO (L1) is “pointy”
- Ridge (L2) is “smooth”



Pros and Cons of LASSO/Ridge

- LASSO
 - Can create sparse vectors
 - Sparse vectors can speed up learning by reducing the number of parameters that contribute to the model (computationally efficient)
 - This makes LASSO good for feature selection/dimensionality reduction (up next)
 - Less sensitive to outliers
 - Chooses arbitrarily between highly-correlated features
 - Example in a furnished home: # of beds vs # of bedrooms
 - Domain knowledge may indicate which feature is actually more helpful
- Ridge
 - Can (empirically) have better predictive performance than LASSO
 - More sensitive to outliers due to squared terms
 - Makes the weights small but not 0

Elastic Net: Best of Both Worlds

- Elastic Net:

$$\hat{w}_{ElasticNet} = \underset{w}{\operatorname{argmin}} MSE(w) + \lambda_1 ||w||_1 + \lambda_2 ||w||_2^2$$

- Finds best λ_1 and λ_2
 - With $\lambda_1 = 0$, just Ridge Regression
 - With $\lambda_2 = 0$, just LASSO Regression



Demo

Break

Get some water, stretch your legs, chat, be back in 10

Dimensionality Reduction

Dimensionality Reduction

- Curse of dimensionality: as you increase the number of features (adding dimensions to the dataset), the volume of the space increases exponentially
 - This means that the data points that you have become scattered sparsely throughout the space, leading you to need more data to fill the space
 - More computation is needed to process the data
 - Models can become too complex, overfitting to noise
 - Models become less interpretable
- Regularization can help, but we can also reduce the amount of regularization needed by reducing the dimensions (# of features) in advance

Dimensionality Reduction

- We could take all of our features and try every subset, testing performance on the validation set
- Question: why not?

Dimensionality Reduction

- We could take all of our features and try every subset, testing performance on the validation set
- Question: why not?
- Answer: how many models would we have to evaluate? 2^D , where D is the total number of features
 - And models will take longer to train the larger the subsets are
 - It becomes computationally infeasible with even small sets of features to choose from (e.g. 100)
- We'll cover three options for dimensionality reduction:
 - Greedy algorithms (forward stepwise, backward stepwise, etc)
 - LASSO Regression
 - PCA

Dimensionality Reduction: Greedy Algorithms

- **Forward Stepwise**
 - Start with no features and iteratively add the features that are the most useful to fit
 - This algorithm never discards features that are already chosen
- **Backwards Stepwise**
 - Start with all features and iteratively remove features least useful to fit
- **Combining forward and backwards steps**
 - Do a forward greedy algorithm that eventually gets features that are no longer as useful to fit

Lots of others!

Ex: Combining forward and backwards steps

Do a forward greedy algorithm that eventually prunes features that are no longer as relevant

Greedy Algorithms Example

Use the forward algorithm to choose two features. Which ones are chosen, and what is the loss on the validation set?

Performance with one feature

Features	Val Loss
# bath	100
# bed	120
sq ft	115

Performance with two features

Features	Val Loss
(# bath, # bed)	155
(# bath, sq ft)	140
(# bed, sq ft)	130

Dimensionality Reduction: Greedy Algorithms

- Forward Stepwise

```
min_val =  $\infty$ 
 $S_0 \leftarrow \emptyset$ 
for  $i \leftarrow 1..k$ :
    Find feature  $f_i$  not in  $S_{i-1}$ , that when combined
    with  $S_{i-1}$ , minimizes the validation loss the most.
     $S_i \leftarrow S_{i-1} \cup \{f_i\}$ 
    if val_loss( $S_i$ ) > min_val:
        break # No need to look at more features
```

- Question: will a greedy algorithm get the best answer?

Dimensionality Reduction: Greedy Algorithms

- Forward Stepwise

```
min_val =  $\infty$ 
 $S_0 \leftarrow \emptyset$ 
for  $i \leftarrow 1..k$ :
    Find feature  $f_i$  not in  $S_{i-1}$ , that when combined
    with  $S_{i-1}$ , minimizes the validation loss the most.
     $S_i \leftarrow S_{i-1} \cup \{f_i\}$ 
    if val_loss( $S_i$ ) > min_val:
        break # No need to look at more features
```

- Question: will a greedy algorithm get the best answers?
- No, it will find greedily optimal answers, which are not necessarily the global optimum

Greedy Algorithms Example

Use the forward algorithm to choose two features. Which ones are chosen, and what is the loss on the validation set?

Performance with one feature

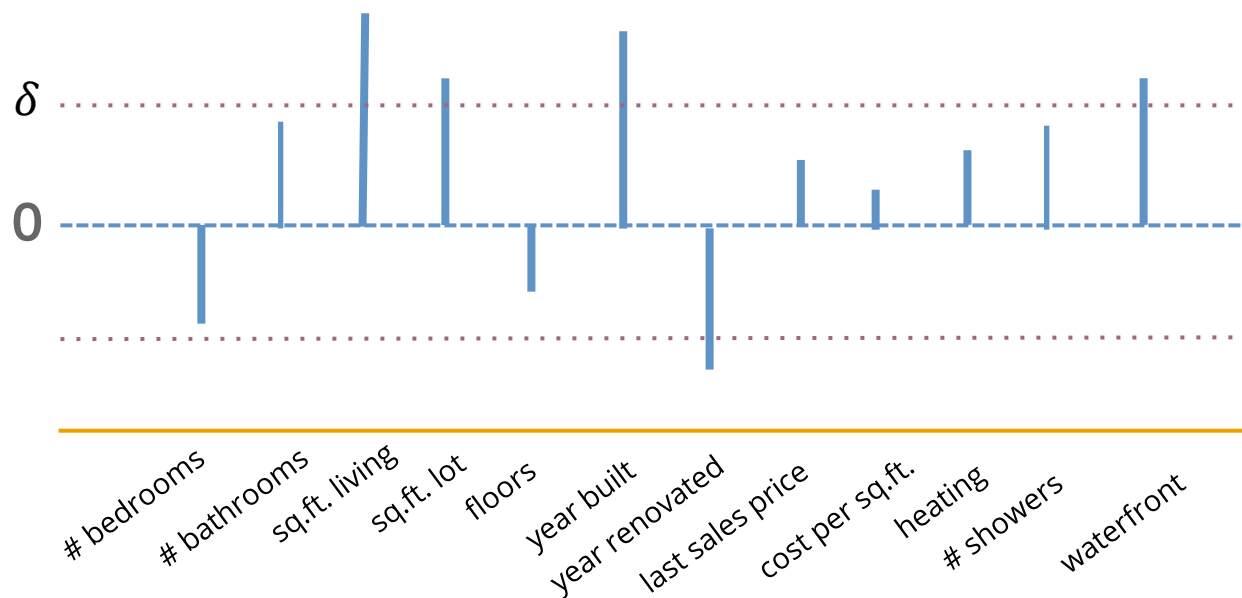
Features	Val Loss
# bath	100
# bed	120
sq ft	115

Performance with two features

Features	Val Loss
(# bath, # bed)	155
(# bath, sq ft)	140
(# bed, sq ft)	130

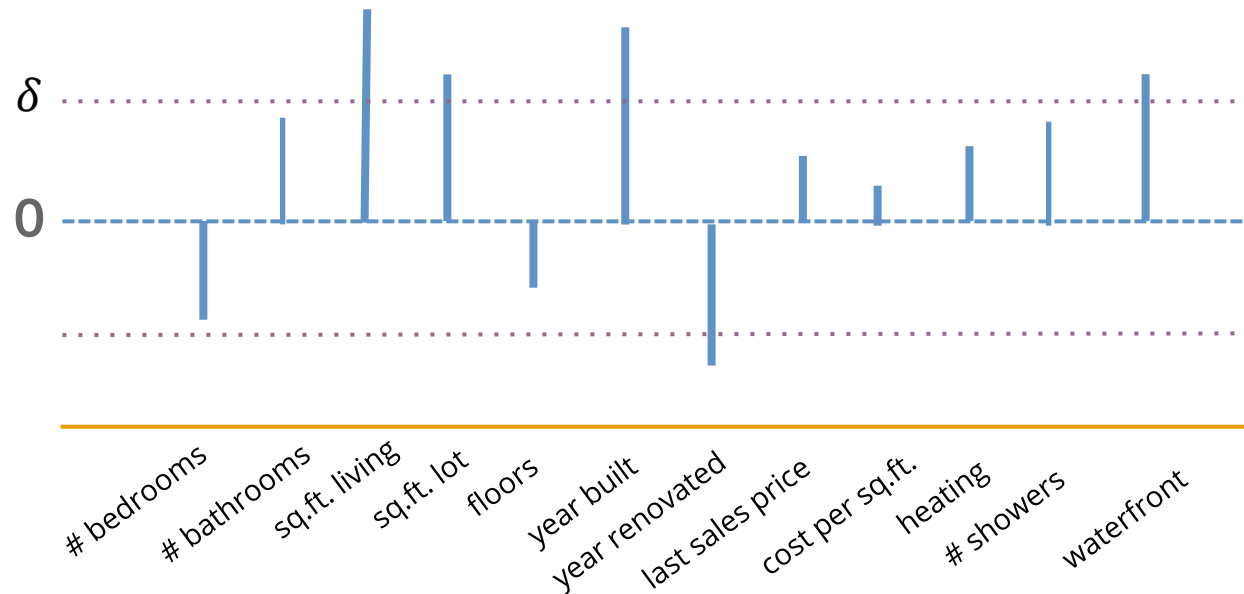
Dimensionality Reduction: Regularization

- We saw two regularization algorithms: Ridge and LASSO
- Can we use Ridge for feature selection?
 - Ridge shrinks coefficients but doesn't set them to 0
 - Can we remove weights that are “small enough”?
 - Discuss: does this work?



Dimensionality Reduction: Regularization

- Problems with using Ridge for feature selection
 - δ is another hyperparameter that needs setting
 - In this example, the chosen delta would get rid of all information about bathrooms
 - If we had removed one of # bedrooms or # showers, the coefficients might have been higher: highly correlated features will make coefficients “share” the weights



Dimensionality Reduction: Regularization

- Problems with using Ridge for feature selection
 - δ is another hyperparameter that needs setting
 - In this example, the chosen delta would get rid of all information about bathrooms
 - If we had removed one of # bathrooms or # showers, the coefficients might have been higher: highly correlated features will make coefficients “share” the weights
 - Luckily, LASSO does this for us!

Dimensionality Reduction: Regularization

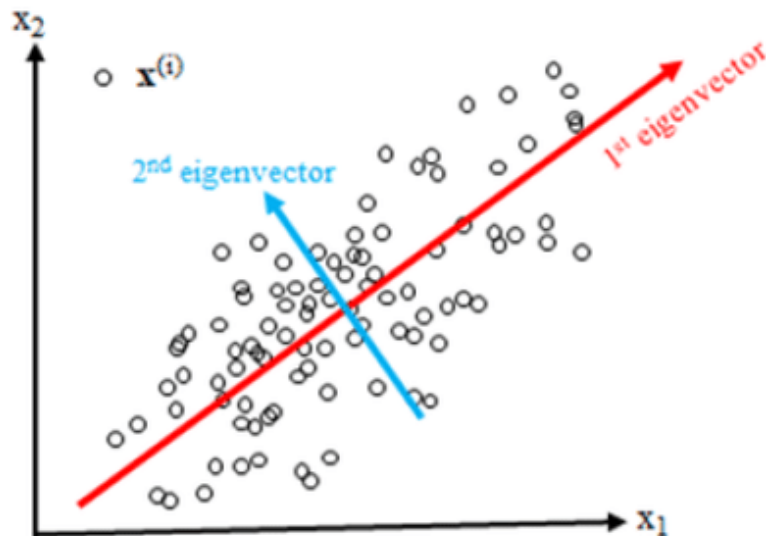
- LASSO for feature selection
 - Use LASSO on a model with **all** features
 - Set λ to get the appropriate complexity (can test performance on validation set)
 - Find the non-zero coefficients and use those features
 - Or, select a subset based on which features are most important using domain knowledge

Dimensionality Reduction: Regularization

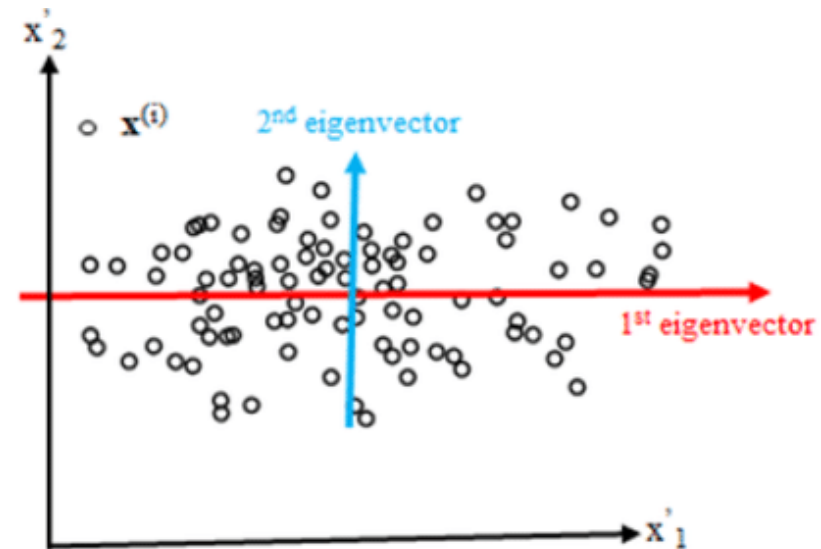
- LASSO adds bias to the model, in order to reduce variance that leads to overfitting
 - Note: Ridge also does this, but not the focus of this dimensionality reduction setting
 - It is often recommended to run LASSO to select features, and then retrain using standard Linear Regression with the selected features
 - This means that coefficients aren't shrunk from their true values, but the model will be less complex than using all features, reducing the need for regularization in learning

Dimensionality Reduction: PCA

- PCA: Principal Components Analysis
- NOT feature **selection**, but feature **extraction**
 - Instead of selecting a subset of the available features, PCA extracts new features that are “combinations” of the original features
 - How? Matrix math (eigenvectors)



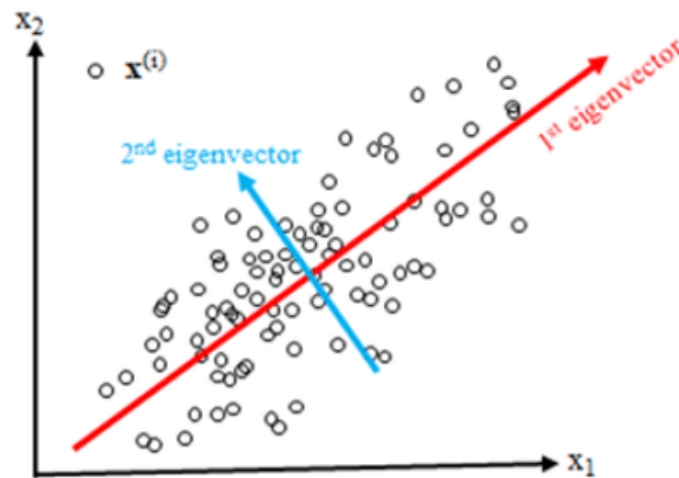
a



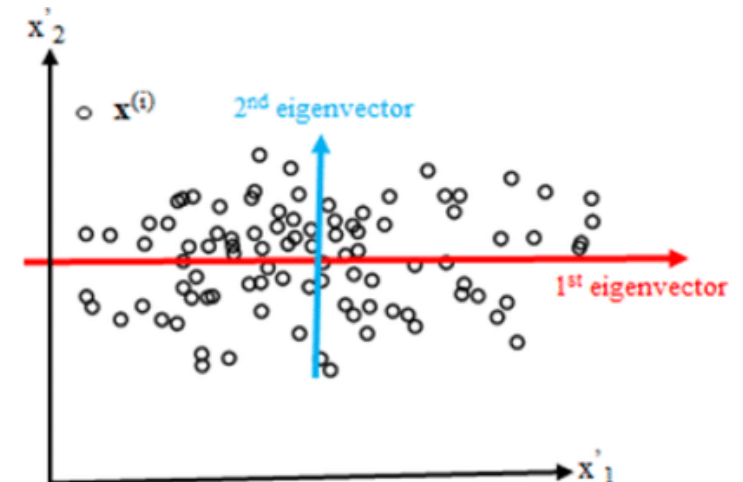
b

Dimensionality Reduction: PCA

- We can create a **covariance matrix** from our features, detailing the variance of each feature and the covariance with each other feature
- The eigenvectors of the covariance matrix:
 - Are orthogonal (at right angles) -> statistically independent
 - Represent the directions of variance in the dataset, ordered by the magnitude of their eigenvalues
- Step one: what do we do with the eigenvectors once we have them?
- Step two: how do we get the eigenvectors?



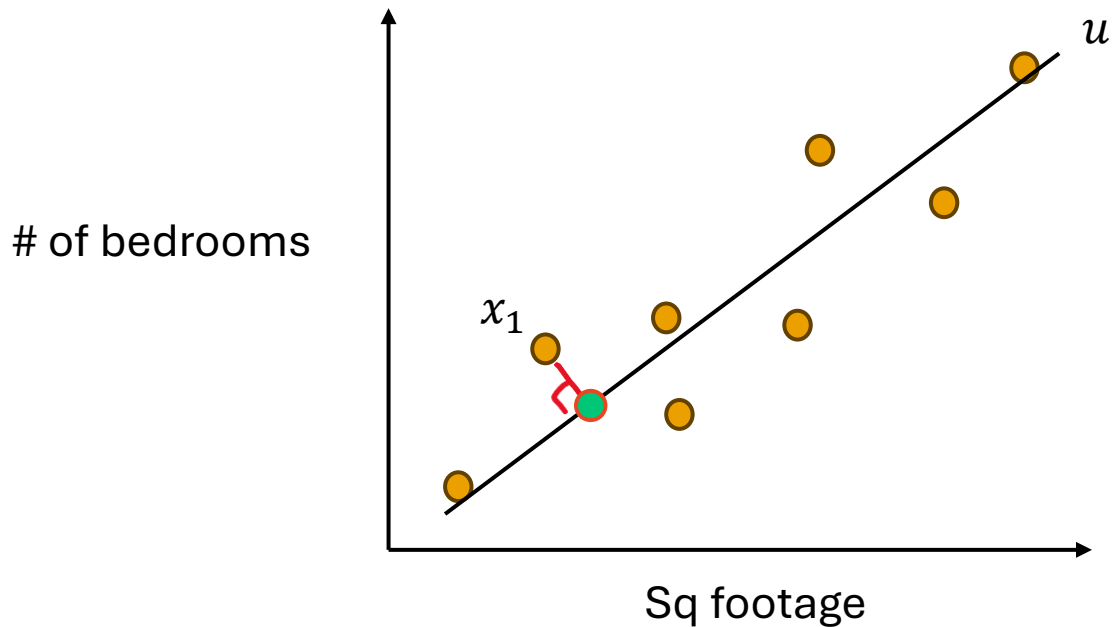
a



b

Linear Projection

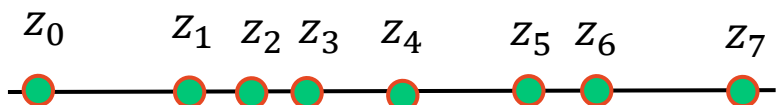
- Let's start with an example of 2-dimensional data, and project it onto a single vector
- In this way, we can represent 2d data as a single number



$$z_i = u^T x_i = \sum_{j=0}^{D-1} u[j] \cdot x_i[j]$$

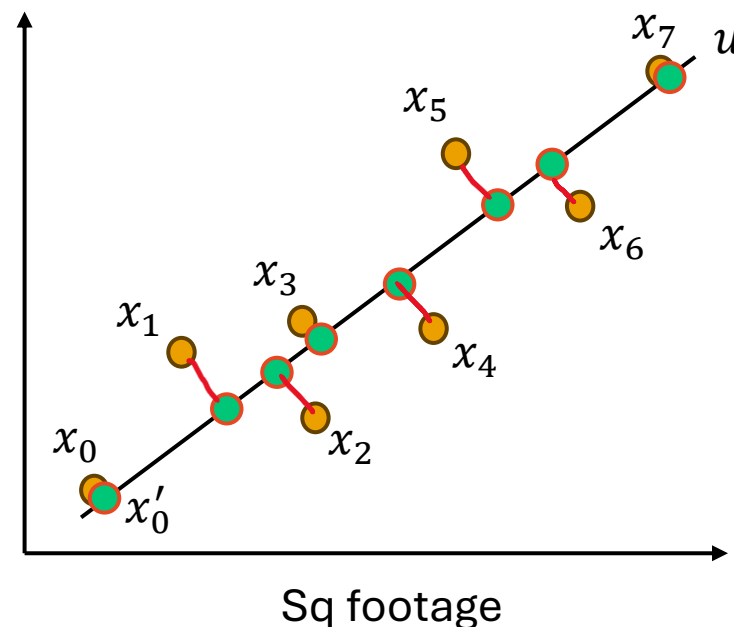
Reconstruction

- To reconstruct the data (only having access to the projected info), we can use the original vector to go back into 2d space
- But there's a reconstruction error: $\|x'_i - x_i\|_2^2$
- Look familiar? It's just the residuals again!
- Using eigenvectors as the projection vectors minimizes reconstruction error



$$x'_i = z_i u$$

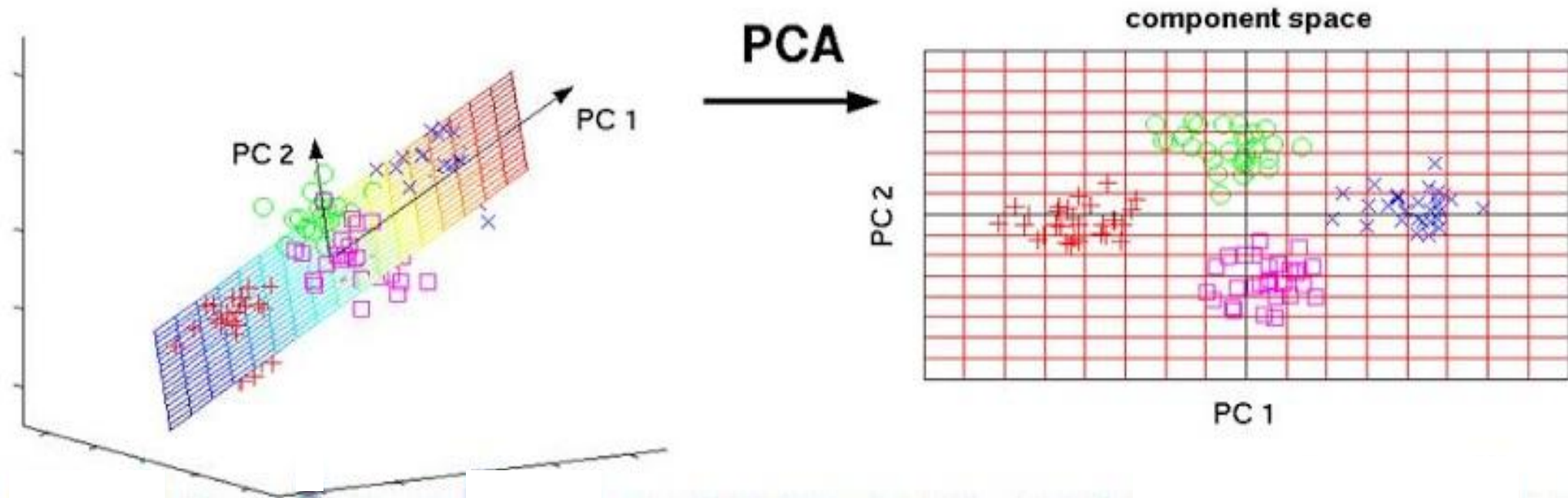
of bedrooms



Disclaimer: this example is hand-drawn and does not represent a real dataset/eigenvector

Reconstruction

- PCA lets you represent points that were originally D dimensions as k dimensions instead, where $k \ll D$ (much less than D)



Example Linear Projection/Reconstruction

- Compute the 2D coordinates of $x_i = [0, 2, 5, 1, 3]$ using the vectors $u_0 = [0.5, 0, -0.5, 0, 0.5]$ and $u_1 = [-0.5, 0, 0.5, 0, 0.5]$. Then compute the reconstruction error.

$$\begin{aligned} z_i &= [?, ?] \\ x'_i &= [?, ?, ?, ?, ?] \\ \|x'_i - x_i\|_2^2 &= ? \end{aligned}$$

$$z_i = u^T x_i = \sum_{j=0}^{D-1} u[j] \cdot x_i[j] \quad x'_i = z_{i,0} u_0 + z_{i,1} u_1$$

Example Linear Projection/Reconstruction

- Compute the 2D coordinates of $x_i = [0, 2, 5, 1, 3]$ using the vectors $u_0 = [0.5, 0, -0.5, 0, 0.5]$ and $u_1 = [-0.5, 0, 0.5, 0, 0.5]$. Then compute the reconstruction error.

$$z_{i,0} = \sum_{j=0}^4 u_0[j] \cdot x_i[j] = 0.5 * 0 + 0 * 2 - 0.5 * 5 + 0 * 1 + 0.5 * 3 = -1$$

$$z_{i,1} = \sum_{j=1}^5 u_1[j] \cdot x_i[j] = -0.5 * 0 + 0 * 2 + 0.5 * 5 + 0 * 1 + 0.5 * 3 = 4$$

$$z_i = [-1, 4]$$

$$z_i = u^T x_i = \sum_{j=0}^{D-1} u[j] \cdot x_i[j] \quad x'_i = z_{i,0} u_0 + z_{i,1} u_1$$

Example Linear Projection/Reconstruction

- Compute the 2D coordinates of $x_i = [0, 2, 5, 1, 3]$ using the vectors $u_0 = [0.5, 0, -0.5, 0, 0.5]$ and $u_1 = [-0.5, 0, 0.5, 0, 0.5]$. Then compute the reconstruction error.

$$z_i = [-1, 4]$$
$$x'_i = z_{i,0}u_0 + z_{i,1}u_1 = [-0.5, 0, 0.5, 0, -0.5] + [-2, 0, 2, 0, 2] = [-2.5, 0, 2.5, 0, 1.5]$$
$$\|x'_i - x_i\|_2^2 =$$

$$z_i = u^T x_i = \sum_{j=0}^{D-1} u[j] \cdot x_i[j] \quad x'_i = z_{i,0}u_0 + z_{i,1}u_1$$

Example Linear Projection/Reconstruction

- Compute the 2D coordinates of $x_i = [0, 2, 5, 1, 3]$ using the vectors $u_0 = [0.5, 0, -0.5, 0, 0.5]$ and $u_1 = [-0.5, 0, 0.5, 0, 0.5]$. Then compute the reconstruction error.

$$z_i = [-1, 4]$$

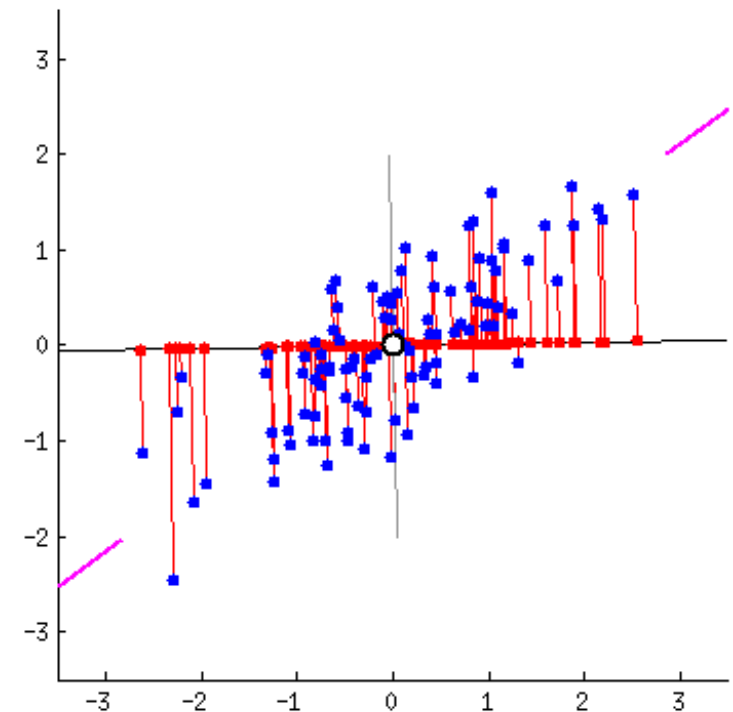
$$x'_i = [-2.5, 0, 2.5, 0, 1.5]$$

$$\begin{aligned}\|x'_i - x_i\|_2^2 &= (-2.5 - 0)^2 + (0 - 2)^2 + (2.5 - 5)^2 + (0 - 1)^2 + (1.5 - 3)^2 \\ &= 6.25 + 4 + 6.25 + 1 + 2.25 = 19.75\end{aligned}$$

$$z_i = u^T x_i = \sum_{j=0}^{D-1} u[j] \cdot x_i[j] \quad x'_i = z_{i,0} u_0 + z_{i,1} u_1$$

Choosing the best vectors for linear projection

- The reconstruction error on the example was pretty bad; you want to make sure you're selecting vectors that will minimize reconstruction error!
- You want to pick the vector(s) along the directions of most variation in the data
- Luckily, the eigenvectors of the covariance matrix will do that for us!
- And the eigenvalues will tell us how “important” each eigenvector is



PCA Algorithm

Input data: Matrix X , the $n \times D$ matrix containing your data and features

1. Center the data: Subtract the mean from each row

$$X_c = X - \bar{X}[0:D-1]$$

2. Compute covariance matrix Σ

$$\Sigma[t, s] = \frac{1}{n} \sum_{i=0}^{n-1} X_c[i, t] X_c[i, s] = \frac{X_c^T X_c}{n}$$

3. Find the eigenvectors and eigenvalues of Σ

1. Select k eigenvectors ($u_0, \dots, u_{k-1}$) with largest eigenvalues: these are your **principal components**

4. Project data: project the data onto the **principal components**

$$z_i[0] = u_0^T X_c[i] = u_0[1] X_c[i, 1] + \dots + u_0[D] X_c[i, D]$$

...

$$z_i[k-1] = u_{k-1}^T X_c[i] = u_{k-1}[1] X_c[i, 1] + \dots + u_{k-1}[D] X_c[i, D]$$

Reconstructed Data

- Using the principal components (the selected eigenvectors) and the projected data, you can reconstruct the data and measure the reconstruction error

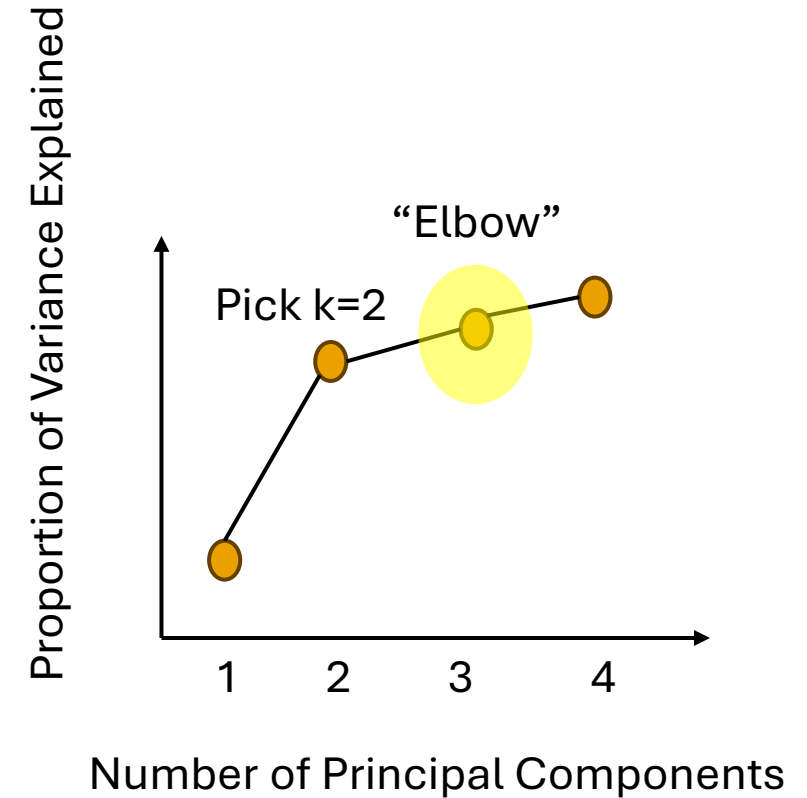
$$x'_i[0:D-1] = \bar{X}[0:D-1] + \sum_{j=0}^{k-1} z_i[j] u_j$$

Choosing k

- How do you choose how many principal components to use?
 - Option 1: select a number of components that are easy to visualize (2 or 3, usually)

Choosing k

- How do you choose how many principal components to use?
 - Option 2: Choose based on the *proportion of variance explained* by the principal components
 - The *proportion of variance explained* by a set of principal components is represented by the sum of the eigenvalues of the kept eigenvectors over the sum of the eigenvalues of all vectors, as the eigenvalues represent the “importance” of each eigenvector
 - Can choose an amount of variance to capture (e.g. 95%), or plot the proportion of variance explained and choose the number at the “elbow”



Choosing k

- Remember: principal components will always be chosen in the order of decreasing eigenvalue magnitude
 - If you only use one principal component, it should be the eigenvector with the highest eigenvalue

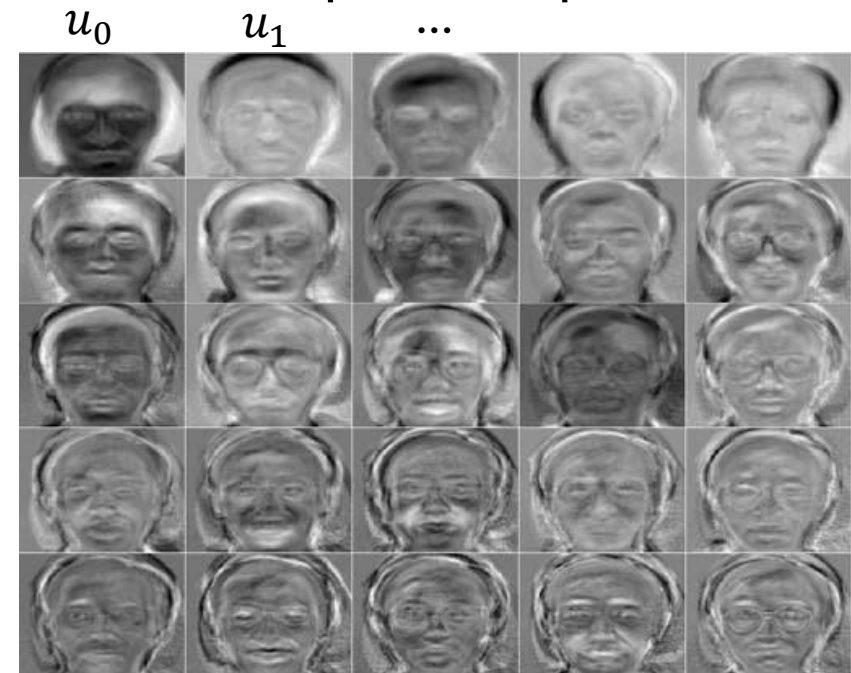
PCA Example Application

- Image data is high-dimensional in pixel space (a 16x16 pixel image gives 256 inputs)
- This example shows PCA applied to a dataset of face images. The top 25 eigenvectors were used to represent faces

Input Data



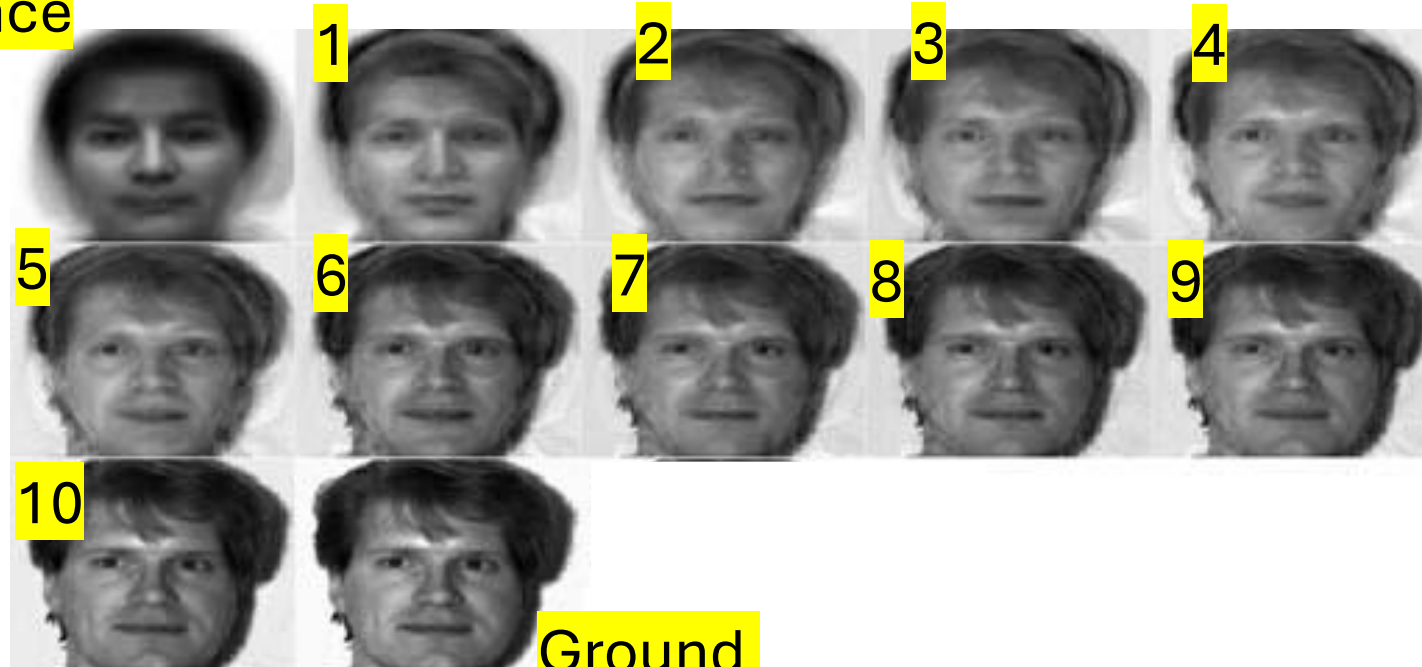
Principal Components



PCA Example Application

- Faces could be reconstructed using eigenvectors: this example shows 1-10 principal components being used

Average Face



Ground
Truth

Pros and Cons of Shown Dimension Reduction Methods

- Greedy algorithms
 - Pros: easy to implement, fast
 - Cons: doesn't find global optimum, just a greedy solution
- LASSO
 - Pros: keeps features interpretable (just selects a subset of existing features)
 - Cons: chooses arbitrarily between highly correlated features, computationally expensive for large datasets
- PCA
 - Pros: faster than LASSO, doesn't completely remove features, ensures independent variables
 - Cons: less interpretable features